Task 2, Part C

In Part C, you must create a pdf format document called written-tasks.pdf, which explains the impact of only allowing each letter to appear once in each word (regardless of how many times the letter appears on the board). Your answer must state an upper-bound on the time complexity reflecting the impact of this change, with each term used explained clearly.

In order to avoid trivial answers, you must assume the board could be extended arbitrarily to higher dimensions (e.g. 5x5 and beyond) and that the alphabet used could increase in size (e.g. the maximum length of a word is not 26 letters).

To quantify and denote the change proposed within part C, it is first pivotal to understand the program's original time complexity and when and how it degrades to a worst-case situation. If we consider,

- A boggle board of dimension **n x n**
- an alphabet of length **L**
- an average word length of **l**
- **d** number of words in a given dictionary
- A branching factor of **b** (i.e. number of connections for each cell)

How will an algorithm like this perform arbitrarily, considering the operations performed

- Construction of the prefix tree
- A depth-first search-like approach to searching the board

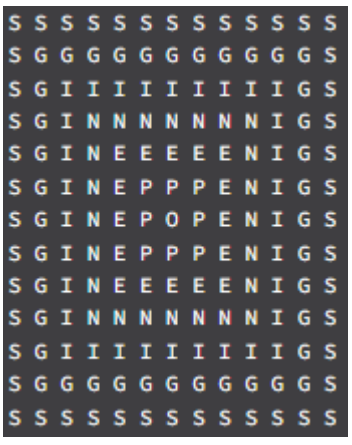
These respective operations can be performed in,

- O(d x l)
- Consider a single cell DFS search; time complexity is O(V + E), which in the case a highly connected graph like a boggle board, where $E \geq V \approx$ O(E), where edges is given by $E = b^{avgdepth} \leq b^{min(n^2,l)}$. Further considering a board of size of n x n, n^2, DFS calls will be performed for which,

  $$number\ of\ edges\ considred\ is, (n^2) * \left(b^{\min(n^2,l)}\right) \Longrightarrow O(\ (n^2) * \left(b^{\min(n^2,l)}\right))$$

  This is an upper bound on the respective operation. Note in the case of the standard board, **b** will be **8**

Now, it becomes apparent by this that the program will degrade to its worst-case when the average depth of each search is greater; that is, the average search length degrades to n^2 per search. Or, in the case of many repeated characters that form words,
Like the example provided here from the ed discussion



Where essentially, as a consequence of repeated characters within the board, the same paths are explored multiple times, degrading to an un-optimal case

Now, let's consider the implications of the suggested restrictions. If we eliminate the consideration of repeated strings, we not only limit the algorithm's asymptotic growth in the case L < n^2 , but also reduce the branching factor of the search algorithm in any boggle board that contains repeated characters. This change allows for the formation of intuitive partitions from the set that optimizes the graph in the worst cases, potentially improving the algorithm's performance.
First, consider that n and L grow arbitrarily large, given no repeated characters, and the maximum possible word length becomes bounded by L where L < n^2 . This imposed restriction also means that as opposed to being proportional to the word length as l approaches L, the search is proportional to the number of unique characters within the board, as repeated characters can not be considered within the search. Further, as characters are partitioned off, the branching factor for each subsequent step in the current search will decrease; though an exact value is hard to quantify, an upper bound for this improvement can be denoted,

$$\begin{aligned} &number\ of\ edges\ considred\ is, \\ &\quad where\ b\ is\ the\ branching\ factor \\ &\quad = 8, and\ \ b^* \ is\ the\ current\ branching\ factor \\ &\quad \leq 8, the\ current\ (n^2) * \left((b^*)^{\min(n^2, L)}\right) \Longrightarrow O(\ (n^2) * \left((b^*)^{\min(n^2, L)}\right)) \end{aligned}$$

This partitioning can be optimised further by utilising extra memory within the tree; suppose each tree cell has a visited matrix that denotes all the places in the board visited by itself and each other character in its prefix, a large partition is created. This can be done based on the idea that a character can only be used once for each word. Consider 'openigs' from the graph above. This partition implies that when searching for an 'e following the prefix 'o–p', any e previously considered to follow this suffix is invalid as all the suffixes from this position would have been found for this prefix would have already been found. Extending this again, it also implies that for that 'e' node, no, e's, o's, or p's have to be considered as the next node, thus creating a large partition in this particular case. And while, again, this is hard to quantify, its branching factor b** could be thought of as b** <= b* <= b.